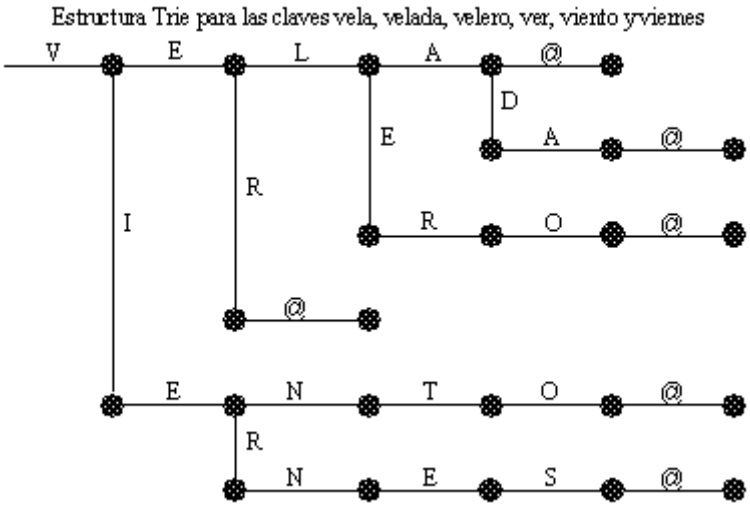
4.5.- Arboles Multicamino Trie

Una estructura trie es esencialmente un árbol n-ario o multicamino. El término trie viene de la palabra "re**trie**val" que significa recuperar o recuperación. Dicha estructura se introdujo en el año 1960 por Fredkin.
A diferencia de las estructuras vistas anteriormente que basaban las búsquedas en la comparación entre claves, los trie (conocidos también como árboles digitales) utilizan la representación de las claves como secuencias de dígitos o caracteres alfabéticos.
Los Trie son parte de los árboles multicamino y son útiles cuando la búsqueda de alguna clave se necesita hacer caracter por caracter. No son más que árboles de prefijos.
Cada nodo en un trie es un estado al que se llega recorriendo los arcos correspondientes a la secuencia de caracteres que este estado representa. Para evitar confusiones entre claves similares como "si" y "sin" usaremos un caracter especial '@' para marcar el final de una cadena de caracteres, de manera que ningún prefijo de una palabra puede ser en sí otra palabra. En otras palabras, en un trie, un camino completo desde la raíz hasta una hoja descendiente corresponde a una clave dentro del conjunto de claves posibles.
La altura del árbol está relacionada directamente a la cantidad de caracteres de las palabras ingresadas, así la palabra con mayor cantidad de caracteres será quién determine cuál es la altura correspondiente.

Veamos un ejemplo de Arbol Multicamino Trie:



4.5.1.- Representaciones de Arboles Multicamino Trie

Existen varias formas de representar un Arbol Trie, entre las más conocidas tenemos:

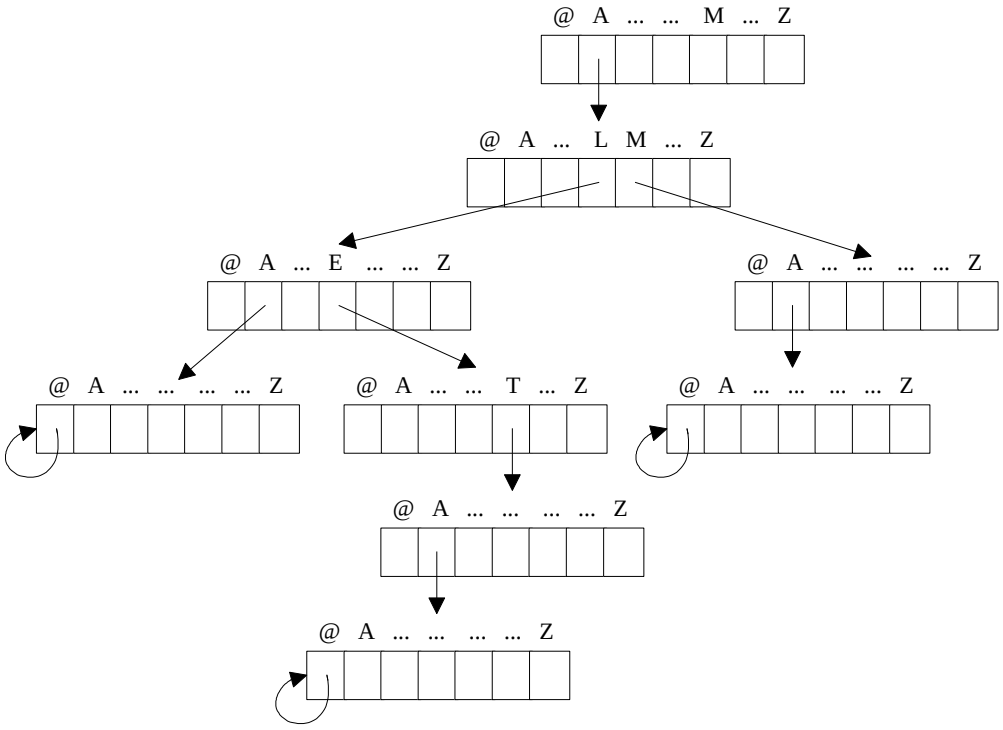
- Mediante matrices

Claves: hija, hijo, hoja, lata, latita, leo y liz.

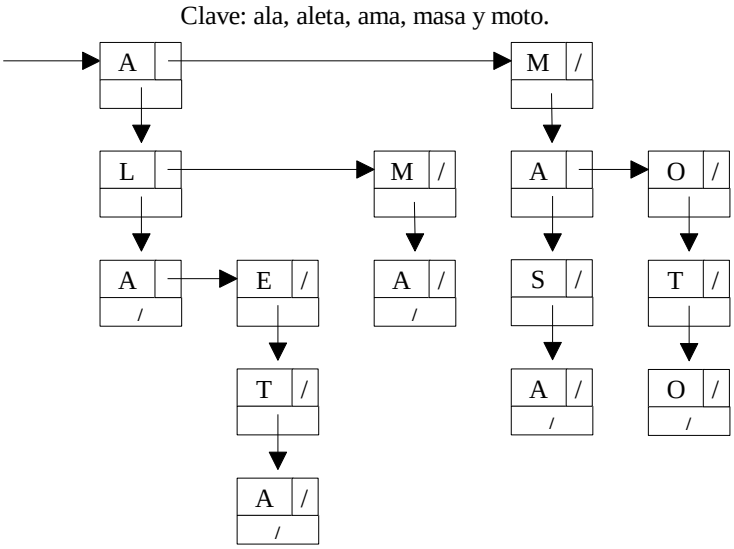
	@	A	B	C	D	E	F	G	H	I	J	K	L	...	O	...	T	...	Z
0									1				12						
1										2					8				
2											3								
3		4													6				
4	5																		
5																			
6	7																		
7																			
8											9								
9		10																	
10	11																		
11																			
12		13				21				24									
13																	14		
14		15								17									
15	16																		
16																			
17																	18		
18		19																	
19	20																		
20																			
21															22				
22	23																		
23																			
24																			25
25	26																		
26																			

- Mediante arreglos enlazados

Claves: ala, aleta y ama.



- Mediante listas enlazadas



4.5.2.- Búsqueda, Inserción y Borrado en Arboles Multicamino Trie

En representaciones de arreglos enlazados cualquier operación es más rápida que listas enlazadas, pero consume mucho espacio (proporcional al número de estados multiplicado por el número de símbolos).
Cuando la representación es por medio de listas enlazadas, puede ser ineficiente en el tiempo de búsqueda de una clave cuando hay muchos arcos que parten del mismo nodo.

4.5.3.- Implementación de Arboles Multicamino Trie

Adaptación de los algoritmos de TRIE propuestos en el libro Estructuras de Datos y Algoritmos de Aho, Hopcroft y Ullman.

```
class Trie
        {
        TrieNode root;
        final int end=26;
        final int begin=0;

        Trie()
                {
                root=null;
                }

        void insertWord(String word)
                {
                TrieNode t;
                int i, pos;
                if(root==null)
                        root=new TrieNode(end);
                t=root;
                i=0;
                //pasar word a minúscula o mayúscula
                while (i<word.length())
                        {
                        pos=getPosition(word.charAt(i));
                        //falta validar posición pos
                        if(t.getValueAt(pos) == null)
                                t.setValueAt(pos,new TrieNode(end));
                        t=t.getValueAt(pos);
                        i++;
                        }
                t.setValueAt(begin, t);
                }
```

```
boolean searchWord(String word)
	{
	TrieNode t=root;
	int i=0;
	if(t==null)
		return false;
	int pos;
	//ver el tema de minúsculas y mayúsculas
	while (i<word.length())
		{
		pos=getPosition(word.charAt(i));
		//falta validar posición pos
		if(t.getValueAt(pos)!=null)
			{
			t=t.getValueAt(pos);
			i++;
			}
		else
			return false;
		}
	if(t.getValueAt(begin)==t)
		return true;
	else
		return false;
	}
//dos formas de obtener la posición de una letra (elegir una)
//primera implementación de getPosition
static int getPosition(char c)
	{
	switch(c)
		{
		case '@': return 0;
		case 'a': return 1;
		case 'b': return 2;
		case 'c': return 3;
		case 'd': return 4;
		case 'e': return 5;
		case 'f': return 6;
		case 'g': return 7;
		case 'h': return 8;
		case 'i': return 9;
		case 'j': return 10;
		case 'k': return 11;
		case 'l': return 12;
		case 'm': return 13;
		case 'n': return 14;
		case 'o': return 15;
		case 'p': return 16;
		case 'q': return 17;
		case 'r': return 18;
		case 's': return 19;
		case 't': return 20;
		case 'u': return 21;
		case 'v': return 22;
		case 'w': return 23;
		case 'x': return 24;
		case 'y': return 25;
		case 'z': return 26;
		}
	return -1;
	}

//segunda implementación de getPosition
static int getPosition(char c)
	{
	Character caux=new Character(c);
	if(c=='@')
		return 0;
	else
		if(c>='a' & c<='z')
			return caux.hashCode()-96;
		else
			return -1;
	}
```

```
//Mediante arreglos enlazados
class TrieNode
        {
        TrieNode[] chars;

        TrieNode(int end)
                {
                chars=new TrieNode[end+1];
                }

        TrieNode getValueAt(int pos)
                {
                //falta validar posición
                return chars[pos];
                }

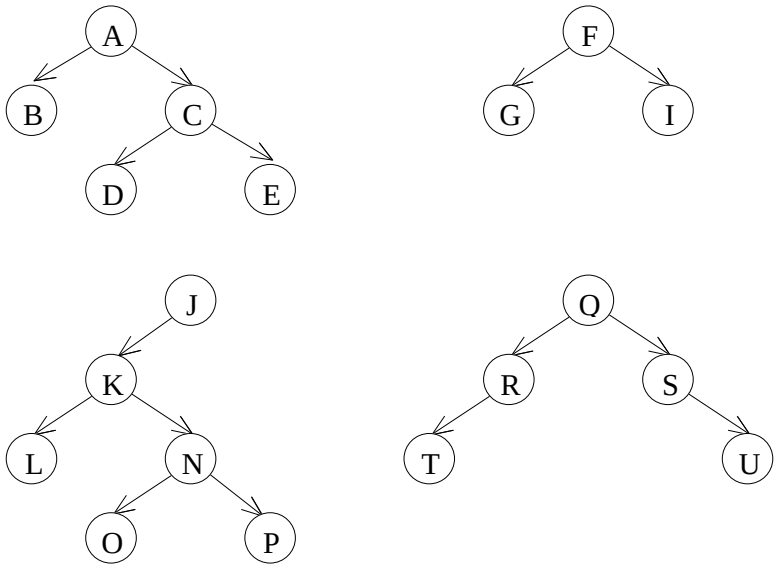
        void setValueAt(int pos, TrieNode newNode)
                {
                //falta validar posición
                chars[pos]=newNode;
                }
        }
//Mediante listas enlazadas
class TrieNode
        {
        char character;
        TrieNode right; //abecedario
        TrieNode down; //palabra
        .
        .
        }
```

5.- BOSQUES

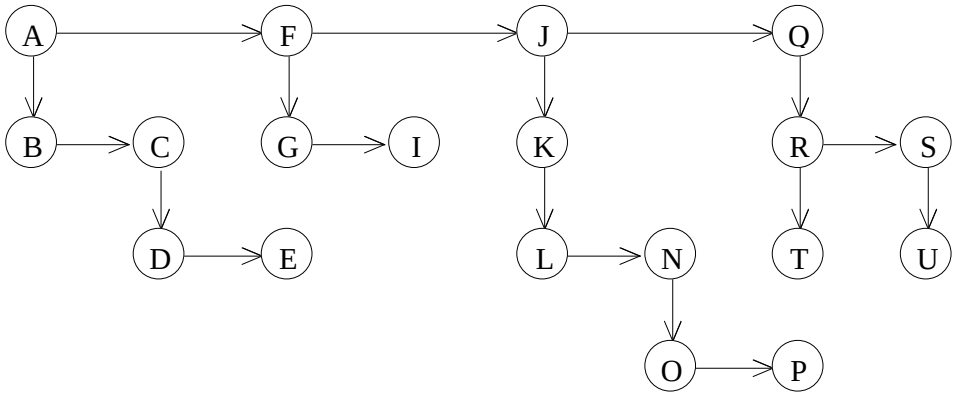
Un bosque representa un conjunto normalmente ordenado de uno o más árboles generales, que normalmente tienen algo en común. Es posible representar un bosque mediante un árbol binario. Los pasos que deben aplicarse para lograr la conversión son:

1. Deben enlazarse en forma horizontal las raíces de los distintos árboles generales.
2. Deben enlazarse los hijos de cada nodo en forma horizontal (los hermanos).
3. Deben enlazarse en forma vertical el nodo padre con el hijo que se encuentra más a la izquierda. Además, debe eliminarse el vínculo de ese padre con el resto de sus hijos.
4. Debe rotarse el diagrama resultante, aproximadamente 45 grados hacia la izquierda y así se obtendrá el árbol binario correspondiente.

a) Bosque compuesto de cuatro árboles



b) Arbol luego de aplicar el primer, segundo y tercer paso



c) Arbol binario luego de aplicar el cuarto paso

